Multi-Layer Perceptron (MLP) for Classification

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

iris = load_iris()
X, y = iris.data, iris.target
scaler = StandardScaler()
X = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = Sequential([
    Dense(16, activation='relu', input_shape=(4,)),
    Dense(8, activation='relu'),
    Dense(3, activation='softmax')
])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

history = model.fit(
    X_train, y_train, epochs=50,
    batch_size=5, validation_split=0.1, verbose=0
)

loss, accuracy = model.evaluate(X_test, y_test, verbose=0)
print("Test Accuracy:", round(accuracy * 100, 2), "%")
print("Test Loss:", round(loss, 4))

plt.plot(history.history['accuracy'], label='Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
```

```
plt.legend()
plt.show()

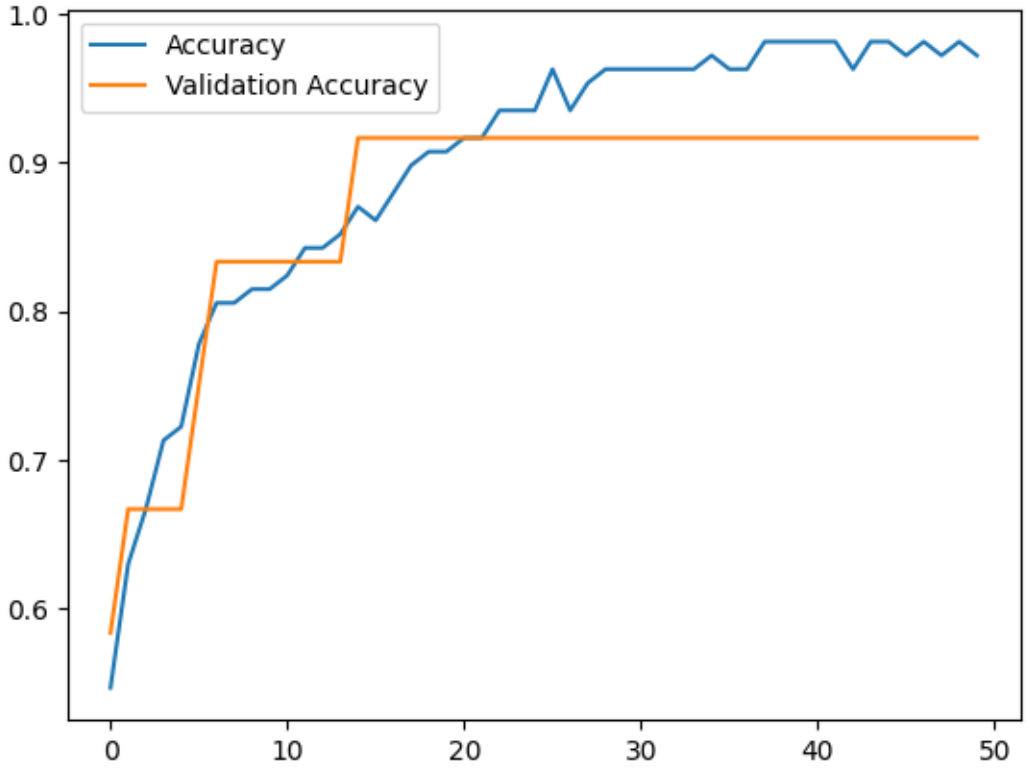
plt.plot(history.history['loss'], label='Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend()
plt.show()

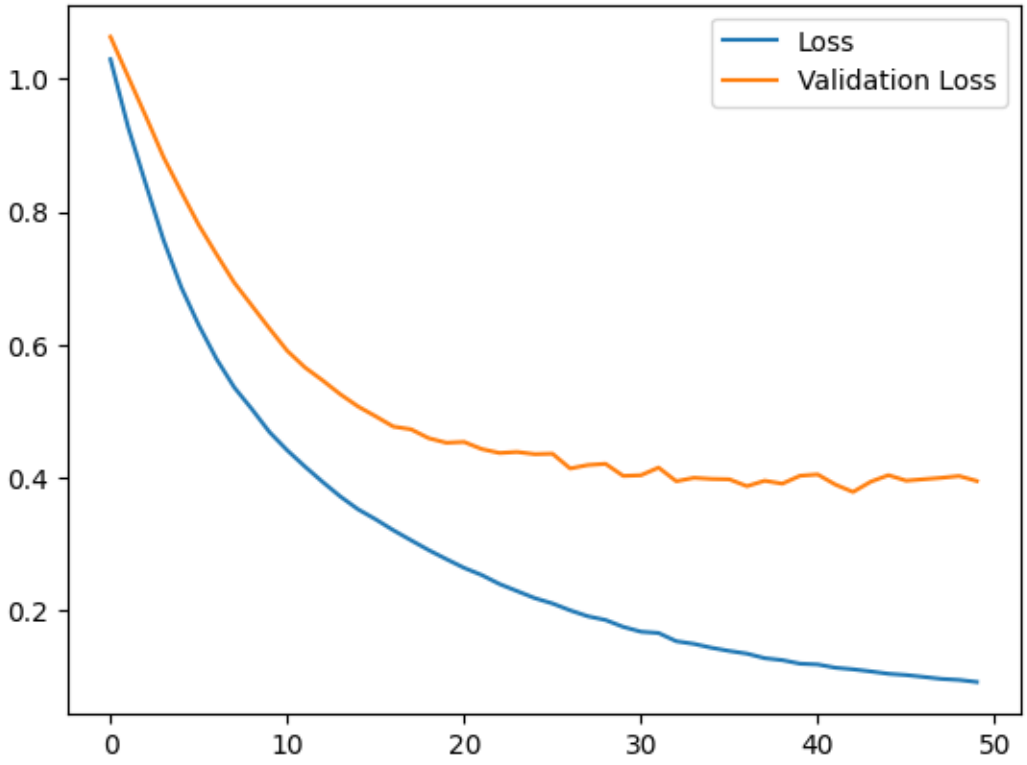
values = np.array([[5.1, 3.5, 1.4, 0.2]])
prediction = model.predict(scaler.transform(values), verbose=0)
print("Predicted Iris Species:", iris.target_names[np.argmax(prediction)])
```

OUTPUT:

```
/usr/local/lib/python3.13/dist-
packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass
an `input_shape`/`input_dim` argument to a layer. When using
Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)

Test Accuracy: 100.0 %
Test Loss: 0.0654
```






```
Predicted Iris Species: setosa
```